

Memory Efficient Doubly Linked List
–Insert At Position and Time Complexity
–Part 3

***When if(curr == NULL) → Condition is True. [When Position
is out of bounds]***

let , pos = 6

```
while (curr != nullptr && i < pos - 1)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
    i++;
}
```

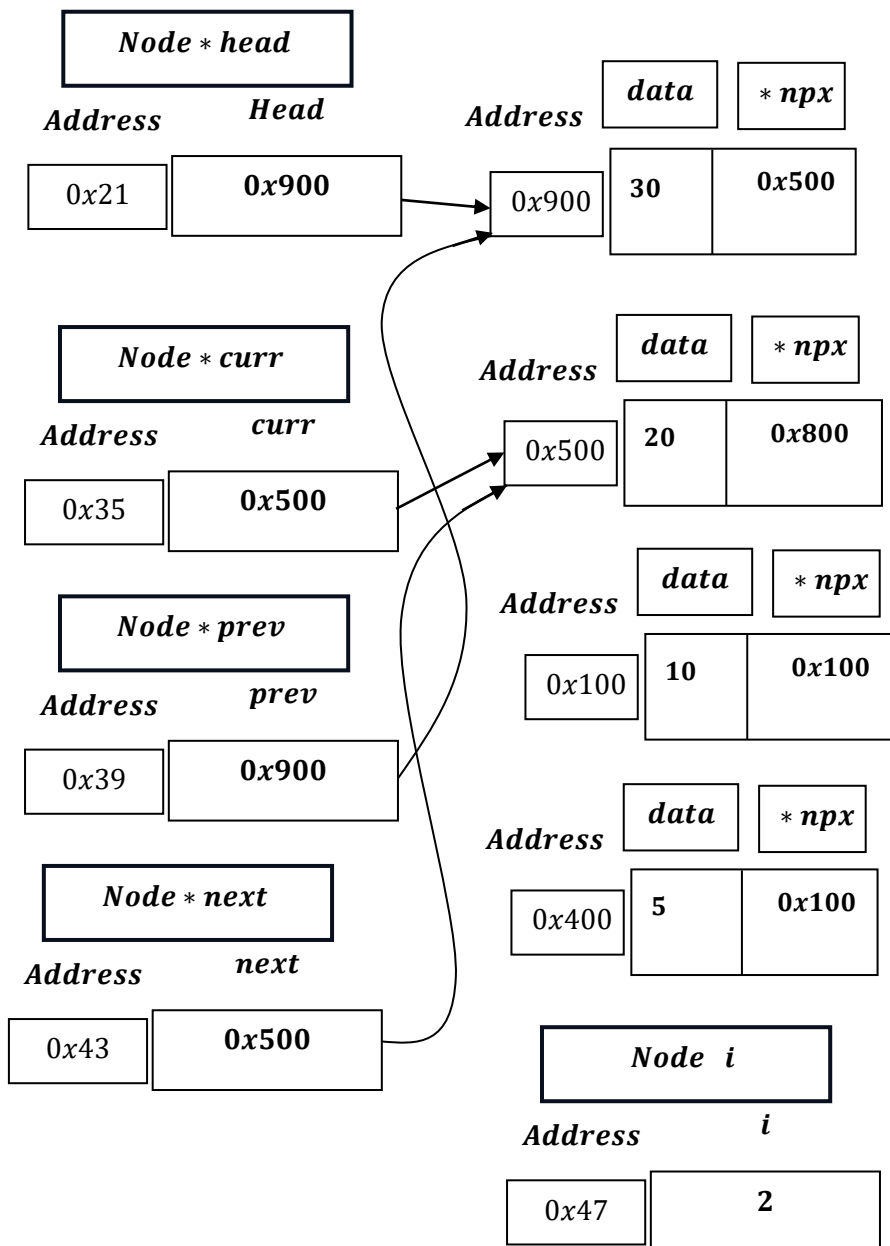
$curr(0x900) \neq 0x000$ And $i < (6 - 1 = 5) \Rightarrow 1 < 5$ is true:

$next = XOR(0x000, curr \rightarrow npx \Rightarrow 0x500) = 0x500$.

$prev = curr = 0x900$;

$curr = next = 0x500$;

$i = i + 1 = 1 + 1 = 2$;



$curr(0x500) \neq 0x000$ And $i < (6 - 1 = 5) \Rightarrow 2 < 5$ is true:

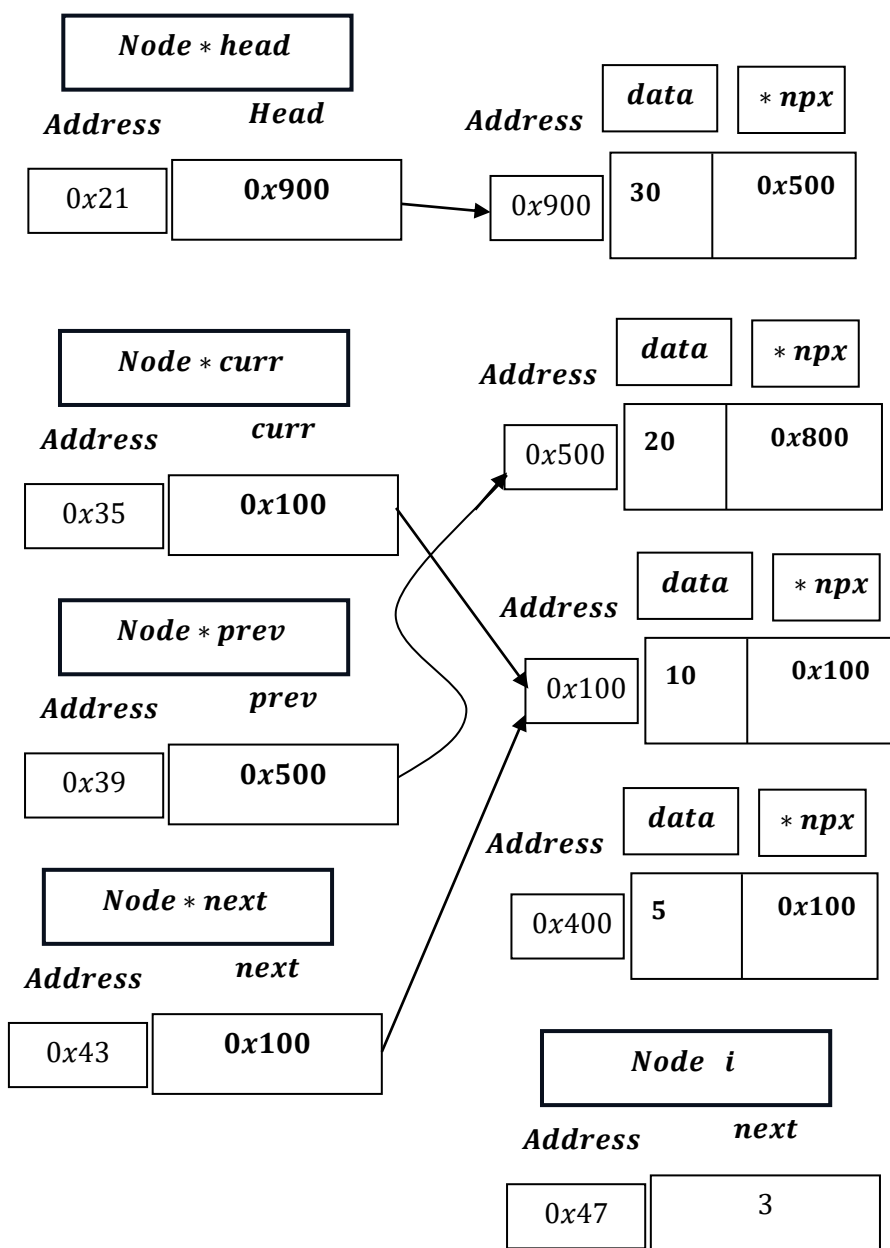
$next = XOR(0x900, curr \rightarrow npx \Rightarrow 0x800)$

$= 1001_2 \oplus 1000_2 = 0001_2 = 0x100$

$prev = curr = 0x500;$

$curr = next = 0x100;$

$i = i + 1 = 2 + 1 = 3;$



$curr(0x100) \neq 0x000$ And $i < (6 - 1 = 5) \Rightarrow 3 < 5$ is true:

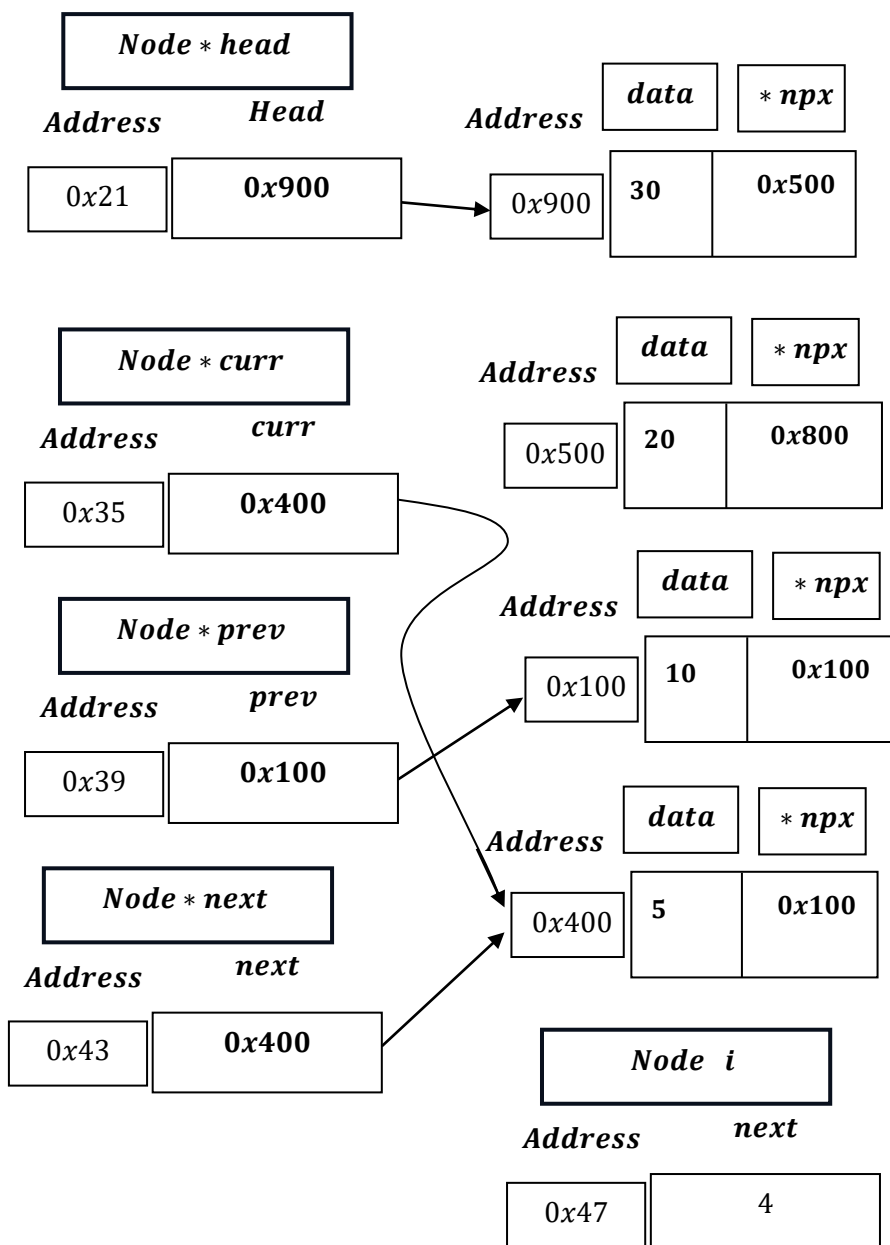
$next = XOR(0x500, curr \rightarrow npx \Rightarrow 0x100)$

$= 0101_2 \oplus 0001_2 = 0100_2 = 4_{10} = 0x400$

$prev = curr = 0x100;$

$curr = next = 0x400;$

$i = i + 1 = 3 + 1 = 4;$



$curr(0x400) \neq 0x000$ And $i < (6 - 1 = 5) \Rightarrow 4 < 5$ is true:

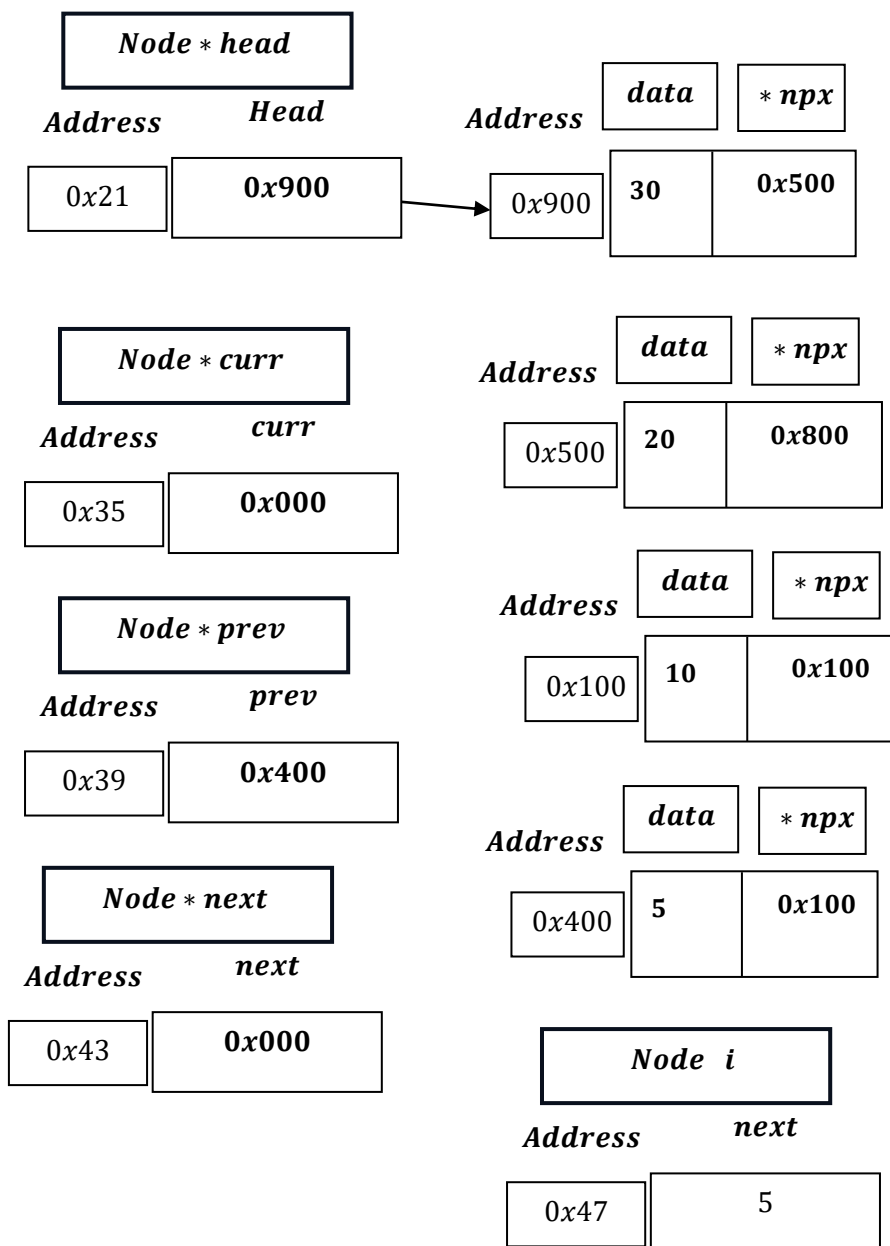
$next = XOR(0x100, curr \rightarrow npx \Rightarrow 0x100)$

$= 0001_2 \oplus 0001_2 = 0000_2 = 0_{10} = 0x000$

$prev = curr = 0x400;$

$curr = next = 0x000;$

$i = i + 1 = 4 + 1 = 5;$



$\text{curr}(0x000) \neq 0x000$ is false And $i < (6 - 1 = 5) \Rightarrow 5 < 5$ is false.

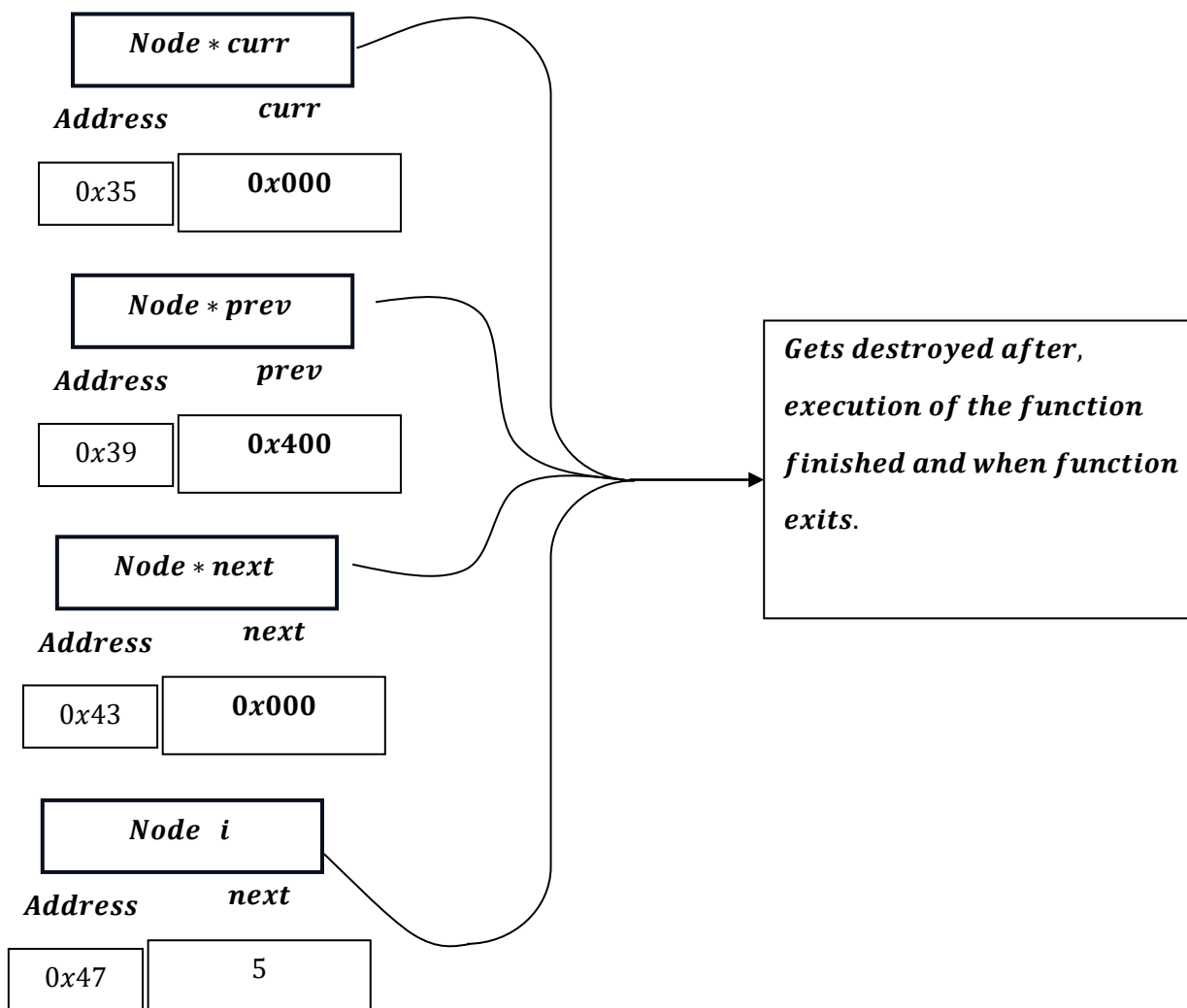
Hence , loop exits.

```
if (curr == nullptr)
{
    cout << "Position out of bounds!" << endl;
    return;
}
```

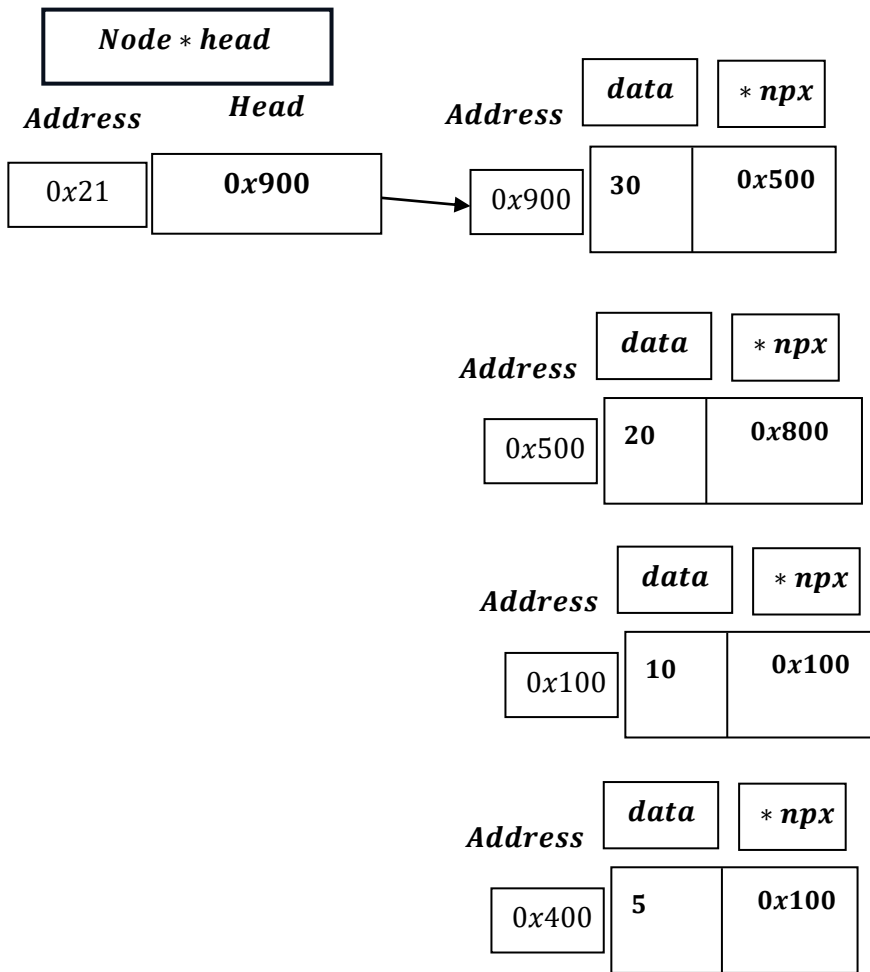
As, $\text{curr} = \text{NULL}$, the it prints "Position out of bounds!"

Then, return void and exits from the function.

As , the function exits :



And we are left with:



Time Complexity of Insert At Position

```
void insertAtPosition(int data, int pos)
{
    if (pos < 1)
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1)
    {
        insertAtBeginning(data);
        return;
    }

    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    int i = 1;

    while (curr != nullptr && i < pos - 1)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
        i++;
    }

    if (curr == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }

    // We are inserting AFTER 'curr'.
    // Let 'next' be the node currently after 'curr'.
}
```

Time Complexity Analysis:

- Invalid position check:** $O(1)$
- Insert at beginning:** $O(1)$
- Pointer initialization:** $O(1)$
- While loop condition:** $O(1)$
- While loop body:** Runs n times at worst time complexity.
- Out of bounds check:** $O(1)$


```

next = XOR(prev, curr->npx); —————→ O(1)

Node *newNode = (Node *)malloc(sizeof(Node)); —————→ O(1)
newNode->data = data; —————→ O(1)

// Linkage Logic:
// 1. Setup newNode: npx = curr ^ next
newNode->npx = XOR(curr, next); —————→ O(1)

// 2. Update curr: npx was (prev ^ next), now becomes
(prev ^ newNode)
curr->npx = XOR(prev, newNode); —————→ O(1)

// 3. Update next (if it exists): npx was (curr ^
nextNext), now becomes (newNode ^ nextNext)
if (next != nullptr)
{
    Node *nextNext = XOR(curr, next->npx);
    next->npx = XOR(newNode, nextNext);
} —————→ O(1)

cout << data << " inserted at position " << pos << "."
<< endl; —————→ O(1)
}

```

Best Case

Best Case : Absolute Best Case $\rightarrow POS < 1, = O(1)$.

Also, pos == 1 also Best Case but not Absolute Best Case.

Worst Case

The loop takes `n` times at worst case ,when position `pos` is out of bounds. $\approx O(n)$.

Average Case

→ If the position is 1, the loop's inner statement runs 0 time.

→ If the position is 2, the loop's inner statement runs 0 time.

→ If the position is 3, the loop's inner statement runs 1 times.

→ If the position is 4, the loop's inner statement runs 2 times.

.

.

→ If the position is $(N + 1)$, the loop's inner statement runs : $N - 1$ times ,

As when $i = N < N$ becomes false ,hence no execution of loop's inner statements.

$$\text{Hence, } \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}$$

Now, as we take all the possibilites then ,we take the best case also i.e.

$O(1)$, when position = 1 and 2. Considering `T(P)` is time complexity of position `P` .

$$T(P) = \begin{cases} 1 & , \text{when } P = 1. \\ 1 & , \text{when } P = 2. \\ P - 2 \text{ or } N - 1 & , \text{when } P = 3, 4, \dots, N + 1. \end{cases}$$

Now, If there is `N` no. of nodes there we can insert data at $N + 1$ positions.

For position 1, the probability of insertion is $\frac{1}{N + 1}$.

For position 2, the probability of insertion is $\frac{1}{N + 1}$.

.

.

For position $N + 1$, the probability of insertion is $\frac{1}{N + 1}$.

Hence probability of insertion will always remain $\frac{1}{N + 1}$.

$\therefore$ Average Time Complexity is : probability of insertion $\times$ (number of possible iterations + constant time complexities for left over positions) .

$$= \left(\frac{1}{N + 1} \times 1 + \frac{1}{N + 1} \times 2 + \dots + \frac{1}{N + 1} \times N - 1 \right) + \frac{1}{N + 1} \times 1 + \frac{1}{N + 1} \times 1$$

$$= \frac{1}{N + 1} \times \{(1 + 2 + 3 + \dots + N - 1) + 1 + 1\}$$

$$= \frac{1}{N + 1} \times \left\{ \sum_{i=1}^{N-1} i + 2 \right\}$$

$$= \frac{1}{N + 1} \times \left(\frac{N(N - 1)}{2} + 2 \right)$$

$$= \frac{N(N - 1)}{2(N + 1)} + \frac{2}{N + 1}$$

$$= \frac{N^2 - N}{2N + 2} + \frac{2}{N + 1}$$

Applying Big Theta in the equation:

$$= \Theta\left(\frac{N^2 - N}{2N + 2}\right) + \Theta\left(\frac{2}{N + 1}\right)$$

$$= \Theta\left(\frac{N^2 - N}{2N + 2}\right) + \Theta\left(\frac{2}{N + 1}\right)$$

Now, taking out the dominant term of numerator and denominator:

- ***In the numerator, $N^2 - N$, the term N^2 grows much faster than N .***

So, for large N , the dominant term is N^2 .

- ***In the denominator, $2N + 2$, the term $2N$ grows much faster than the constant 2. The dominant term is $2N$.***

$$= \Theta\left(\frac{N^2}{2N}\right) + \Theta\left(\frac{2}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{2}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + 2 \times \Theta\left(\frac{1}{N}\right)$$

Now removing the constant `2` we get:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant $\frac{1}{2}$, we get:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ terms, largest growth rate is the result:

$\rightarrow \Theta(N)$ grows linearly with N .

$\rightarrow \Theta\left(\frac{1}{N}\right)$ decreases to a constant, i.e. when N grows larger, $\frac{1}{N}$ approaches to 0.

This we can prove by limit and continuity also:

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0, \text{ where } c \text{ is constant}\right].$$

As $\Theta(N)$'s growth rate is higher, therefore, average case is $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

Note, by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N(N-1)}{2(N+1)} + \frac{2}{N+1} \leq c_2 \times N$$

Average Case Time Complexity[Most Simplest Alternative Apporach]:
(Simplest approach)

For position 1, the probability of insertion is $\frac{1}{N+1}$.

For position 2, the probability of insertion is $\frac{1}{N+1}$.

.

.

For position $N+1$, the probability of insertion is $\frac{1}{N+1}$.

***Hence probability of insertion will always remain $\frac{1}{N+1}$,
for position 1 to $N+1$.***

Average case complexity : Probability of insertion $\times$ $\left(\begin{array}{l} \text{Summation} \\ \text{of number of} \\ \text{Position.} \end{array} \right)$

$$= \frac{1}{N+1} \times \left(\sum_{i=1}^{N+1} i \right)$$

$$= \frac{1}{N+1} \times \left(\frac{1}{2} (N+1) ((N+1) + 1) \right) \left[\text{As, } \sum_{k=1}^n k = \frac{1}{2} (n)(n+1) \right]$$

$$= \frac{1}{N+1} \times \left(\frac{(N+1)(N+2)}{2} \right)$$

$$= \frac{N+2}{2}$$

$$= \frac{N}{2} + \frac{2}{2}$$

$$= \frac{N}{2} + 1$$

Applying Big Theta Θ we get:

$$= \Theta\left(\frac{N}{2}\right) + \Theta(1)$$

$$= \frac{1}{2} \times \Theta(N)$$

$$= \Theta(N)$$

Hence, Average Time Complexity is : $\Theta\left(\frac{N}{2} + 1\right)$ or $\Theta(N)$.

Now,

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N}{2} + 1 \leq c_2 \times N$$

<i>Memory Efficient Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Insert At Position</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$
